Category 4 — Substrate Authoritative for "What Rules Apply": Standalone Treatment of Rule-Set Authority in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: 4 May 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the fourth of the five authoritative-state categories named in the source paper's substrate-as-source-of-truth commitment — substrate authority over the orchestration rule set, the architectural answer to "what rules apply" — as a standalone architectural commitment with independent operational content, separable from the four sibling categories with which it composes.

Abstract

The CKS pattern's substrate-as-source-of-truth commitment names five categories for which the substrate is architecturally authoritative: what is the case, what was decided, what is in conflict, what rules apply, and who has what authority. The integrating frame and the first three categories are formalized in their own derivation notes. This note formalizes the fourth — substrate authority over the orchestration rule set — as having independent architectural content that can be defended, implemented, and tested separately from the others. The motivation is concrete: deployments under pressure to integrate with enterprise governance platforms, rule engines, or policy-management infrastructure routinely produce architectures in which orchestration rules are present in substrate but external systems are treated as the authoritative rule registry. Such architectures may satisfy the related commitment that rule authoring is a design-time governance moment producing substrate content while still failing the source-of-truth commitment that substrate is architecturally authoritative for the rule set itself. The note states what rule-set authority requires, distinguishes it from four adjacent patterns commonly conflated with it, names the failure modes that violate it specifically, and provides an operational test for whether a given system implements the commitment.

1. Why Category 4 needs to be formalized as standalone

The substrate-as-source-of-truth commitment names five categories of authoritative state. The parent foundational note formalizes the joint commitment, an integrating-frame note articulates the five-category structure as a composite, and three sibling notes formalize Categories 1, 2, and 3 — substrate authority for "what is the case," "what was decided," and "what is in conflict." This note formalizes Category 4 — substrate authority for "what rules apply" — as having independent architectural content with particular weight on substrate authority over the *complete* orchestration rule set, including rule content, authoring provenance, applicability scope, and version history.

The motivating cases are deployments where orchestration rules exist in substrate but external systems are treated as authoritative for the rule registry itself. Examples include substrates that record rule content while external policy engines are queried for which rules currently apply; substrates that carry rule definitions while external configuration stores are authoritative for rule applicability scope; substrates that record human-authored rules while external systems generate runtime rule variants treated as authoritative for execution; and substrates that maintain rule history while external rule-management dashboards are treated as the official rule registry. Each pattern fragments authority over the rule set: the substrate is rule-resident but not rule-authoritative, and the operational consequences of the difference are real.

The standalone treatment performs an architectural foreclosure. The commitment to substrate authority for the rule set forecloses architectures in which substrate is treated as a working record while external rule engines, policy-management platforms, or governance dashboards hold authority over which rules exist and what they specify. A system that places authority over the rule set in any external surface is not CKS-coherent on the rule-set axis, regardless of how robustly it satisfies the other four categories or the related rule-authoring commitment.

Category 4 is the orthogonal counterpart of the rule-authoring moment formalized in a separate decomposition note from the moment-axis decomposition of human-governed authority. That note commits to rule authoring as a design-time governance moment producing orchestration rules as substrate content. Category 4 commits to substrate being architecturally authoritative for the resulting rule set. The two compose: rules must be human-authored substrate content at design time AND the substrate must be the architectural source of truth for the rule set. An implementation that satisfies the first but fails the second has substrate-resident rules that are not substrate-authoritative — the rules are in substrate, but external systems are consulted as the authoritative answer for which rules exist and what they specify. Naming Category 4 as standalone is what makes the orthogonality describable.

Category 4 is also load-bearing for the AI-as-substrate-mediator commitment, specifically the property that LLM writes operate under orchestration rules with rule references carried as part of the standard provenance metadata. Category 4 makes the substrate authoritative for the rules the LLM operates under and that the provenance references; without it, those rule references could authoritatively resolve to rules held in external systems, breaking the substrate-only-paths property the source paper's path-retraceability commitment requires.

2. The Category 4 commitment, defined precisely

A system satisfies Category 4 when the substrate is the architectural source of truth for the orchestration rule set the deployment maintains. The commitment has four operational components.

(a) The substrate is the architectural answer to rule-set queries. For queries about orchestration rules — which rules govern this content, what does this rule specify, when was this rule authored and by whom, what is the rule's applicability scope — the architectural answer is the substrate's record of the rule together with its authoring provenance. Other systems may be consulted for rule-execution performance, for visualization, or for runtime indexing; the substrate's answer is the authoritative answer for what the rule is and which rules apply.

(b) The rule set is substrate content in full. The substrate carries the complete set of orchestration rules the deployment maintains — rule content, authoring provenance, applicability scope, and version history. There are no selective subsets, no summary representations standing in for full rule content, and no external-only rule records that the substrate does not carry.

(c) New rules and rule modifications are committed to substrate first. When humans author a rule under the rule-authoring commitment, or modify an existing rule, the change is committed to substrate with the standard provenance fields — writer attribution (which human authored or modified the rule), timestamp, antecedent reference (what substrate content the rule was authored to address), and rationale where applicable. External rule-management systems, if present, update downstream from substrate rather than the other way around.

(d) Disagreements between substrate and external sources are resolved by substrate. When the substrate's record of a rule and external sources' records disagree on rule content, applicability scope, or version, the substrate prevails. If an external rule engine has a rule that does not match substrate, the architectural commitment is to update the engine to match substrate, not to update substrate to match the engine.

The four components together define Category 4 architecturally. A system satisfying fewer than four cannot reliably claim substrate authority over its rule set in the architectural sense, even if its rule management functions effectively in practice.

3. What the commitment does NOT claim

Stating precisely what Category 4 does not claim is what keeps the standalone treatment from drifting into a stronger position than the source paper supports.

It does not claim authority over rule-execution mechanisms. The substrate is authoritative for which rules exist and what they specify. Runtime rule-execution infrastructure — rule engines, evaluators, policy frameworks, decision services — is operational and not Category 4 content. Execution behavior may depend on engine choice, runtime conditions, or operational optimization; the substrate's authority is over rule content and rule applicability, not over execution dynamics.

It does not require all logic-like content to be a rule. Computational logic embedded inside a cell is cell-internal and not orchestration content; deployment scripts, operational scripts, and data-processing logic outside the orchestration framework are not Category 4 content. The architectural commitment is to orchestration rules specifically — the rules that govern cell-level behavior over substrate content.

It does not claim that rules are interpretable without context. Reading a rule from substrate returns the rule's content; understanding what it does, when it applies, or how it interacts with other rules may require additional context — domain knowledge, sibling-rule interaction, or operational history. The architectural commitment is to the rule being authoritatively recorded, not to its interpretation being automatic.

It does not specify rule expression formats. Rules may be expressed in natural language, in structured DSLs, in code-like notation, or in any other form the deployment supports. The architectural commitment is to rules being substrate content queryable through standard read operations, not to a specific expression format.

It does not require external rule-management systems to be absent. Deployments may have rule-management interfaces, rule-design tools, rule-testing frameworks, and policy-visualization dashboards. The architectural commitment is that such systems are derivative — they may add operational value but cannot substitute for substrate authority over the rule set.

It does not specify rule retention duration. Rules persist in substrate as substrate content; how long deployments retain old rule versions, deprecated rules, or rule change history is a deployment concern. Category 4 authority holds for as long as the substrate carries the rules; the duration is set elsewhere.

4. What the commitment is NOT

Four adjacent patterns are commonly conflated with Category 4. Each is a real and reasonable commitment in some other architecture; naming what Category 4 is *not* prevents the misreading.

Not rules-as-code in external repositories. Some implementations store rules as code in version-control repositories, with the repository's commit history treated as the authoritative source. The repository carries rule code, version history, and authoring metadata; substrate may carry rule references or summaries. This pattern fails Category 4 because authority over the rule set lives in the code repository, not in substrate. Substrate-only paths break — the architectural answer to "which rules apply" requires consulting an external repository.

Not rule-management platforms as authoritative. Some implementations use specialized rule-management platforms — governance suites, policy-management applications, regulatory compliance dashboards — as the authoritative rule registry. The platform carries rule content, applicability, and version history; substrate is operationally downstream. This pattern fails Category 4 because the architectural authority is in the external platform; substrate becomes a working surface that mirrors what the platform decides.

Not policy engines as authoritative. Some implementations use policy engines, rule engines, or constraint solvers as the authoritative source for which rules apply and how they evaluate. The engine

carries rules and evaluates them at runtime; substrate may carry inputs and outputs but not the rules themselves, or may carry rules whose authority is the engine's enrolled-rule list rather than substrate state. This pattern fails Category 4 because the rules — including which rules apply to which content — are external to substrate.

Not rules-as-configuration in external configuration stores. Some implementations treat rules as configuration data held in external configuration-management systems — config databases, environment-configuration platforms, or feature-flag services. The configuration store carries rule content and applicability; substrate operates over the rules but does not carry them. This pattern fails Category 4 because rules are external configuration rather than substrate content, and the configuration store's update semantics determine the rule set's authoritative state.

5. Why Category 4 is load-bearing for downstream commitments

Category 4 is a load-bearing dependency for several CKS commitments beyond the integrating source-of-truth commitment from which it specializes.

The integrating substrate-as-source-of-truth commitment is the immediate parent. Category 4 covers one of the five categories; without it, the source-of-truth commitment is partial — substrate would be authoritative for state, decisions, and conflicts but not for the rules that govern them.

The rule-authoring commitment depends on Category 4 for execution coherence. That commitment establishes rule authoring as a design-time governance moment producing substrate content. Without Category 4, an implementation could satisfy rule authoring operationally — humans write rules, rules land in substrate — while external systems are treated as authoritative for which rules apply, diluting design-time governance with external runtime authority over the same rules.

The AI-as-substrate-mediator commitment, specifically the property that LLM writes occur under orchestration rules with rule references in provenance, depends on Category 4 for those references to point to architecturally authoritative content. Without Category 4, the mediator's rule references could resolve to rules held outside substrate, breaking substrate-only paths.

The path-retraceability commitment includes "under what authority" as a provenance field. For cell-produced writes, that authority is the orchestration rule under which the cell operated. Category 4 makes substrate the architectural answer to the rule reference, completing the substrate-only-paths chain retraceability requires.

The cost-model commitment for rule-variety cost depends on rules being substrate-resident and substrate-authoritative. Rule-authoring cost amortizes across cell executions because rules are written once at design time and consulted at runtime through substrate reads. Without Category 4, rule cost would be paid against rules whose authority is external, complicating the amortization architecture and weakening the linear-cost commitment's coverage of rule variety.

These dependencies are not introduced by Category 4; they are dependencies the joint source-of-truth commitment carries. Naming Category 4 as standalone makes them attributable to a specific architectural property that downstream notes can reference precisely.

6. Failure modes that violate the commitment

A system can fail Category 4 specifically, even when it satisfies the other four authoritative-state categories and the rule-authoring commitment. Ten failure modes name the most common ways this happens.

(a) Code-repository-primary architectures. Rules are stored as code in version control; the repository is treated as the authoritative source; substrate carries references or derived state.

(b) Policy-engine-primary architectures. A policy engine is treated as the authoritative rule registry; substrate operates over engine outputs but does not carry the rules themselves with substrate-resident authority.

(c) Rule-management-platform-primary. A specialized rule-management application — governance suite, compliance dashboard, regulatory rule manager, or enterprise policy-management product — is treated as the authoritative rule registry; substrate is a working interface that updates the platform downstream. Enterprise-mandated governance platforms commonly drift into this position because they are operationally familiar and often regulatory-mandated for specific governance categories.

(d) Configuration-store-primary. Rules are stored as configuration data in external configuration-management systems; substrate operates over the rules but does not carry them.

(e) Substrate-summary-external-detail. Substrate carries rule summaries; external systems carry full rule definitions. Substrate is summary-authoritative but not detail-authoritative; the architectural commitment to the *complete* rule set is not satisfied.

(f) Rule-versioning-external. Substrate carries the current active rule set; rule version history lives in external systems. Substrate authority is current-version-bound; the complete rule history is split across substrate and external systems.

(g) Runtime-rule-generation. Rules are generated at runtime by external systems based on substrate content; the generated rules are treated as authoritative for that runtime; substrate is read by the generator but the generated rules' authority is external.

(h) Rule-applicability-external. Substrate carries rule content but rule applicability scope — which rules apply to which content — lives in external configuration. The "what rules apply" question requires external resolution; substrate is content-authoritative but not applicability-authoritative.

(i) Rule-deployment-external. Rules are authored in substrate but deployed to external rule engines for execution; the deployment may transform, optimize, or specialize rules in ways that diverge from substrate; the deployed rules are treated as authoritative for execution behavior.

(j) Hot-reload-overrides. External systems can hot-reload rules at runtime, with the hot-reloaded rules treated as authoritative even when they diverge from substrate. The substrate's authority is overridden by runtime updates that flow in from outside.

A system exhibiting any of (a)–(j) does not satisfy Category 4 in the architectural sense, even if its rule-management functions effectively and its other four authoritative-state categories hold.

7. Operational test

A system satisfies Category 4 if and only if all of the following hold at all times during the substrate's existence:

1. Queries about orchestration rules — which rules govern which content, what does each rule specify, when was each rule authored and by whom, what is each rule's applicability scope — are answered authoritatively from substrate.
2. The substrate carries the complete set of orchestration rules the deployment maintains; no selective subsets, no external-only rule definitions, and no rule content held externally with substrate carrying only references.
3. New rules and rule modifications are committed to substrate first, with full provenance; external rule-management systems, if present, update downstream from substrate.
4. When substrate and external sources disagree on a rule's content, applicability scope, or version, the deployment resolves the disagreement in favor of substrate.
5. Rule applicability — which rules apply to which substrate content — is determined from substrate alone; the "what rules apply" question can be answered without consulting external configuration or runtime systems.
6. The rule set is queryable from substrate alone through standard read operations and substrate-only paths.

A system that fails any of (1)–(6) does not satisfy Category 4 in the architectural sense, even when its rule management operates effectively in practice. Such a system may be a useful system, and may be governed in some other sense, but it is not CKS-coherent on the rule-set axis, and downstream work that relies on rule-set authority guarantees should be scoped accordingly.

8. Conclusion

Implementations under pressure to leverage existing rule engines, integrate with enterprise governance platforms, or support sophisticated runtime rule features routinely drift toward rule authority being held in external systems. The drift is steady because rule engines and policy-management platforms are operationally familiar, well-tooled, and often regulatory-mandated for specific governance categories. The drift looks like compatibility — substrate carries rule content, external systems handle execution — but its architectural consequence is that authority over the rule set fragments across substrate and external surfaces.

Implementations that drift away from Category 4 produce systems where substrate is rule-resident but not rule-authoritative. The downstream consequences manifest as governance failures (humans cannot exercise meaningful governance over rules whose authority is held in external rule engines), mediator failures (LLM rule references in provenance point to rules whose authority lives outside substrate, breaking substrate-only paths), retraceability failures ("under what authority" cannot be answered from substrate alone for cell-produced writes), and source-of-truth fragmentation (substrate authoritative for state, history, and conflicts but external systems authoritative for the rules that govern them).

Naming Category 4 as a standalone architectural commitment gives downstream implementers a precise specification of what substrate authority over the rule set requires, independent of how the other four authoritative-state categories or the rule-authoring commitment are handled. A subsequent note formalizes Category 5 (substrate authority for "who has what authority") on the same pattern, and a further note treats the source-of-truth-versus-mirror-of-truth distinction as standalone; together these complete the decomposition the integrating-frame note initiated.

Subsequent work that adopts, extends, or argues against the CKS commitment to substrate authority over the rule set should use "rule-set authority" in the sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Category 4 — Substrate Authoritative for "What Rules Apply": Standalone Treatment of Rule-Set Authority in the Coordination Knowledge Substrate Pattern.* 4 May 2026. ORCID: 0009-0004-8065-3235.